

Gaussian Math Audit



July 16, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	6
Security Model and Trust Assumptions	7
Notes & Additional Information	9
N-01 Missing Documentation	9
N-02 Inaccurate Documentation	10
N-03 Rational-Function Evaluation Logic Duplicated Across cdf, pdf, and inverse_cdf	10
N-04 Rational Coefficients Are Fit to Continuous Error Instead of the Integer Rounding Behavior	11
N-05 inverse_cdf Tail Change Of Variable Is Carried At Output Scale, Capping Tail Accuracy Below Kernel Precision	12
N-06 inverse_cdf Saturation Bound Is Inconsistent With cdf	13
N-07 WAD Constant Uses 10^{36} Scale Inconsistently With Standard Meaning	14
N-08 cdf and pdf Lack a Bit-Exact Regression Test Against Their Offline Codegen Mirror	14
N-09 Documentation Inconsistencies	15
Conclusion	17
Appendix	18
Independent Numerical Verification	18
Issue Classification	21

Summary

Type	Library	Total Issues	9 (9 resolved)
Timeline	From 2026-07-06 To 2026-07-09	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	9 (9 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the Gaussian distribution functions and their offline coefficient-generation pipeline in the [OpenZeppelin/contracts-sui](#) repository at commit [bb2405c](#). The engagement also included a [diff audit](#) of the fixed-point type API covering the changes between base commit [55a7971](#) and the same target commit.

In scope for the full audit were the following files:

```
math
├── fixed_point
│   └── sources
│       └── internal
│           ├── cdf.move
│           ├── cdf_coefficients.move
│           ├── horner.move
│           ├── inverse_cdf.move
│           ├── inverse_cdf_coefficients.move
│           ├── pdf.move
│           └── pdf_coefficients.move
scripts
├── gaussian_codegen
│   ├── cdf
│   │   ├── derive.py
│   │   ├── emit_coefficients.py
│   │   └── validate.py
│   ├── inverse_cdf
│   │   ├── derive.py
│   │   ├── emit_coefficients.py
│   │   └── validate.py
│   ├── pdf
│   │   ├── derive.py
│   │   ├── emit_coefficients.py
│   │   └── validate.py
│   └── shared
│       ├── aaa.py
│       ├── arithmetic.py
│       ├── constants.py
│       ├── move_emit.py
│       └── reference.py
```

In scope for the diff audit were the following modified files:

```
math
├── fixed_point
│   └── sources
│       └── sd29x9
```

```
|   |— conversions.move
|   |— sd29x9.move
|   |— sd29x9_base.move
|— ud30x9
|   |— conversions.move
|   |— ud30x9.move
|   |— ud30x9_base.move
```

System Overview

The `openzeppelin_fp_math` package supplies fixed-point decimal arithmetic for Sui, built around two `u128`-backed decimal types, `UD30x9` (unsigned) and `SD29x9` (signed, two's complement), each carrying nine decimal places to match Sui's native coin precision. Both types already provide arithmetic, comparison, and logarithm operations; this engagement covers the addition of three closed-form statistical functions built on top of them: `cdf` (the standard-normal cumulative distribution function, Φ), `pdf` (the standard-normal density, ϕ), and `inverse_cdf` (the standard-normal quantile, Φ^{-1}). These are exposed as public methods on both `UD30x9` and `SD29x9` through the `sd29x9_base` / `ud30x9_base` modules and their method-alias wrappers, giving any downstream package a way to compute normal-distribution probabilities directly in fixed point, for uses such as z-score-based risk thresholds or margin and liquidation probability checks.

Each distribution function is implemented as a rational-function approximation, $N(z)/D(z)$, evaluated with Horner's method over a shared internal engine, `horner`, which represents intermediate values as sign-magnitude 256-bit accumulators and multiplies through an internal scale finer than the public `10^9` representation (`10^36` for `cdf` / `pdf`, `10^18` for `inverse_cdf`, chosen to avoid rounding artifacts near the tails). `cdf`, `pdf`, and `inverse_cdf` each pair this engine with a dedicated coefficient table module (`cdf_coefficients`, `pdf_coefficients`, `inverse_cdf_coefficients`), all three explicitly marked as generated code that must not be hand-edited. `inverse_cdf` additionally splits its input probability range into a central region and an upper-tail region, evaluating the tail through a logarithm and square-root change of variable to avoid the numeric blow-up as the probability approaches one. All of this internal machinery is package-private; the only functions reachable from outside the package are the six `cdf` / `pdf` / `inverse_cdf` entry points on the two base types, and every one of them is pure, deterministic, and free of on-chain storage.

The coefficient tables themselves are not derived on-chain. They are produced by an offline Python toolkit (`scripts/gaussian_codegen`) that fits a rational approximation to each target function using the AAA (Adaptive Antoulas-Anderson) algorithm against a float64 or arbitrary-precision reference, then quantizes the fitted coefficients and renders them directly into the Move coefficient modules. Continuous integration does not re-run this derivation step. Instead it re-implements the on-chain fixed-point arithmetic in Python and checks the committed coefficients against a high-precision oracle for accuracy, monotonicity, and integer-overflow headroom, and separately checks that the committed Move file is byte-identical to

what the emitter deterministically produces from a committed intermediate fit-output file. This confirms that the shipped coefficients are numerically sound and have not drifted from the recorded fit, but it does not independently reproduce the fit itself.

The diff portion of the engagement covers the base modules for both fixed-point types (`sd29x9_base`, `ud30x9_base`, `sd29x9`, `ud30x9`, and their `conversions` modules) between the prior audit's end commit and the current target commit. The changes wire the new Gaussian functions onto both types. Six example modules were additionally reviewed at a high level, as references for the intended integration patterns rather than as audited code: `token_normalizer`, `amm_quote`, `fee_split`, and `median_oracle` under the `math/core` package's `examples` directory, and `zscore` and `compounding` under the `math/fixed_point` package's `examples` directory. All six are out of scope, and each is explicitly marked as not production ready.

Security Model and Trust Assumptions

Callers of the distribution functions: any package can call `cdf`, `pdf`, and `inverse_cdf` on `UD30x9 / SD29x9` values without restriction; there is no administrative, owner, or otherwise privileged role in this library, and every function is pure and free of side effects. Callers must supply `inverse_cdf` an argument in `[0, 1]` (the `UD30x9` variant is further restricted to `[0.5, 1]`, since that type cannot represent a negative quantile), or the call aborts. By contrast, `cdf` and `pdf` never abort on an out-of-domain input; instead they saturate to their limiting value once `|z|` exceeds a fixed threshold (approximately 6.109 for `cdf`, 6.402 for `pdf`), so a caller that needs to distinguish extreme tail values beyond those thresholds will instead observe a flat, saturated result. The accuracy guarantee attached to all three functions is stated as an absolute error bound on the order of a few parts in `10^9`, not a relative one, so an integration that needs relative precision on very small tail probabilities should not rely on this library for that purpose.

Offline coefficient-generation pipeline: the coefficient tables compiled into `cdf_coefficients`, `pdf_coefficients`, and `inverse_cdf_coefficients` originate entirely from this pipeline, and nothing in the Move code re-derives or independently re-verifies the underlying rational-function fit at runtime beyond the sign and positivity checks described below. The chain from a validated fit to shipped Move source depends on a human running the derivation step correctly and committing its output honestly. Continuous integration checks

that the committed coefficients pass numerical accuracy, monotonicity, and overflow-margin gates, and that the committed Move file matches what the emitter deterministically produces from the committed intermediate fit output, but it does not re-run the fit end to end and compare the result. Independently of the pipeline's own gates, these same three properties (accuracy against a high-precision reference, monotonicity at neighbor resolution, and integer-overflow headroom) were reproduced from scratch during this engagement, by parsing the committed coefficients, re-implementing the fixed-point evaluation independently, scanning every representable input against an `mpmath` 60-digit oracle, and confirming the outputs on the Sui Move VM; the Independent Numerical Verification appendix documents the method and the per-function results. Anyone depending on this library is therefore trusting that the pipeline's committed intermediate artifacts and Move output were produced by a correctly executed, unmodified run of the documented tooling. As a defense against a corrupted or mis-generated table, each on-chain evaluator asserts that its numerator is non-negative and its denominator is strictly positive before dividing; these assertions are documented as unreachable for the shipped coefficients, so their only current effect is that a future corrupted table would cause affected calls to abort rather than silently return a wrong result. The pipeline documents the exact tool versions (Python, `mpmath`, `scipy`, `numpy`) used to produce the currently committed coefficients, but its dependency manifest only pins lower bounds on those libraries, so re-running the derivation on newer library versions could in principle produce a different, independently valid fit rather than reproducing the committed one bit for bit; this is a consideration for future maintenance and for auditing subsequent coefficient changes rather than a defect in the values shipped today.

Notes & Additional Information

N-01 Missing Documentation

Three groups of missing documentation were identified across `horner.move` and the gaussian math evaluators.

The following functions each call into a Horner evaluation and then assert that the resulting numerator is non-negative and the denominator is strictly positive, aborting with `EInternalNumNegative` or `EInternalDenNonPositive` otherwise, without documenting that possibility in a `#### Aborts` section:

- `cdf::cdf_nonneg_raw` and its private `eval_rational` helper
- `pdf::pdf_nonneg_raw` and its private `eval_rational` helper
- `inverse_cdf::inverse_cdf_upper_raw` and its private `eval_rational` helper

None of these six doc comments include an Aborts section, even though the layer immediately above each of them (the corresponding public function in `sd29x9_base` or `ud30x9_base`) documents the same propagated aborts, framed there as defense-in-depth that cannot fire for the shipped coefficients.

Separately, in `horner.move`, the `mag` and `is_neg` accessors carry no doc comment at all, unlike every other function in the module, and the `mag` and `neg` fields of `SignedScaled256` carry no field-level `///` comment either. The explanations for both fields exist, but only in the struct's top-level documentation rather than above each field, which the project style guide requires.

Consider adding a `#### Aborts` section to each of the six evaluator functions listed above, mirroring the phrasing already used one layer up; adding a one-line summary to `mag` and `is_neg`; and moving the field-specific sentences in `SignedScaled256`'s top comment to `///` comments directly above `mag` and `neg`.

Update: Resolved in [pull request #474](#).

N-02 Inaccurate Documentation

Two instances of documentation describing behavior that does not match the underlying code were identified in the gaussian math modules.

- The [overshoot clamp in `cdf::eval\_rational`](#) is accompanied by a comment stating that rounding "can produce `ONE_RAW + 1` raw at `z` just below `max_z`". This does not hold for the coefficients currently committed in `cdf_coefficients.move`: the pre-clamp value at the largest reachable input is exactly one unit below `ONE_RAW`, not above it. More generally, since the denominator `D(z)` remains positive throughout the domain, the sign of the ratio minus one equals the sign of `N(z) - D(z)`, a degree-nine polynomial with no real roots in the domain and a strictly negative value at both endpoints. The clamp branch is therefore unreachable for the shipped coefficients, not merely unobserved to trigger, and the comment's specific illustrative scenario does not hold up under direct evaluation of that polynomial either.
- A [note labeled "Forward-compat"](#) in `cdf.move`'s module documentation describes `pdf` and `inverse_cdf` as modules that will eventually get their own coefficient tables and reuse the `horner` engine unchanged. Both modules have since been implemented exactly as described, and neither carries an equivalent note; the content remains accurate, but the forward-looking framing is now outdated and could mislead a reader unfamiliar with the module's history.

Consider revising the overshoot-clamp comment to state that the branch is currently unreachable for the committed coefficients and exists as defense-in-depth against a future coefficient regeneration while retaining the clamp itself, and rewording or removing the "Forward-compat" note now that the modules it anticipates already exist.

Update: Resolved in [pull request #475](#).

N-03 Rational-Function Evaluation Logic Duplicated Across `cdf`, `pdf`, and `inverse_cdf`

The step that forms the rational-function ratio `N(z) / D(z)` is implemented separately in `cdf`, `pdf`, and `inverse_cdf` as a private `eval_rational` function, and the three copies are structurally identical: each promotes the argument to the accumulation scale, runs `horner_eval!` over the numerator and denominator, asserts `N` is non-negative and `D` is positive, and rounds `N / D` back to the `10^9` scale with a single nearest step. The `inverse_cdf` copy is even documented as "mirroring `cdf::eval_rational`". The only

variations are values that can be parameterized: the accumulation scale (10^{36} for `cdf` and `pdf`, 10^{18} for `inverse_cdf`), the final clamp (to `ONE_RAW`, to `max_z_raw`, or none), and the two per-module error messages.

Duplicated logic of this kind increases the maintenance surface, since a future correction to the rational-evaluation path must be applied consistently in three places and any divergence between the copies can go unnoticed.

Consider extracting a single generic rational evaluator, parameterized by the accumulation scale and the two error constants, that returns the raw `u256` ratio and leaves each caller to apply its own clamp.

Update: Resolved in [pull request #476](#).

N-04 Rational Coefficients Are Fit to Continuous Error Instead of the Integer Rounding Behavior

Each Gaussian function evaluates a fixed-point rational and rounds the result to the nearest step of the 10^9 output grid, for example the CDF [rounds the ratio in a single nearest step](#). The coefficients that feed that rational come from a continuous rational fit that [drives down the worst-case error of the smooth rational against the target function](#). Those two objectives are not the same. Once the output is rounded to the grid, what determines quality is how many inputs round to a value other than the correctly rounded one (the count of mis-rounded outputs), and a fit tuned to minimize the continuous worst-case error is not the fit that minimizes that count. Accepting a slightly larger peak error in exchange for lower average error carries fewer results across a rounding boundary, so re-deriving the same-degree rational directly against the count leaves fewer mis-rounded outputs at identical degree, coefficient storage width, per-call gas, and public API.

Re-fitting for this objective and scoring every representable input against an independent high-precision reference lowers the mis-rounded population substantially. For the CDF (degree 9/9, which saturates to exactly 1 at $z = 6.109410205$), correctly rounded outputs rise from 96.48% to 99.39%, cutting mis-rounded results from 215,010,459 to 37,127,485. The PDF (degree 10/10, which saturates to 0 at $z = 6.402729806$) is [fit the same continuous way](#), and re-deriving it against the count raises correctly rounded outputs from 98.10% to 99.20%, from 121,390,696 down to 51,235,611. The `inverse_cdf` function (the inverse of the standard-normal CDF) is [fit the same way](#) in its two regions. Its tail region carries a separate structural limit, but the coefficient re-derivation on its own lifts its central region (probabilities up to about 0.975) from roughly 95.45% to 98.98% correctly rounded, from about 21,600,000

down to 4,851,915 mis-rounded outputs of the 475,000,000 central inputs. For the CDF and PDF, every error today is already at most one unit in the last place (never worse), so their mis-rounded count is exactly the count of one-unit errors, and the re-fit keeps that property and the existing monotonicity intact.

These tables are the best found by searching the coefficient space directly against the count, not a proven optimum. They show how much error the current fitting method leaves available, and the true minimum at these degrees may be lower still.

Consider re-deriving the shipped coefficient tables against the integer rounding count at the current degrees. The change lives entirely in the codegen: the on-chain evaluator, the `u128` coefficient storage, and the per-call gas are all unchanged, so it is a drop-in table swap.

Update: Resolved in [pull request #483](#).

N-05 `inverse_cdf` Tail Change Of Variable Is Carried At Output Scale, Capping Tail Accuracy Below Kernel Precision

The `inverse_cdf` function (the inverse of the standard-normal CDF) splits its input range into two regions. For probabilities up to about 0.975 it evaluates a rational in $u = p - 0.5$, and for probabilities in the upper tail (close to 1) it evaluates a rational in the change of variable $r = \sqrt{-2 \ln(1 - p)}$, built by [tail_variable_raw](#). That routine brings the logarithm down to the `10^9` output scale and then takes the square root at that same `10^9` scale, rounding down, even though the underlying logarithm kernel produces roughly eighteen significant digits. Carrying `r` at only `10^9` means neighboring tail probabilities, whose true `r` values differ far below the ninth decimal, collapse onto the identical integer `r` and therefore onto the identical output. This is a resolution floor that no choice of coefficients can lift. Of the 25,000,000 tail probabilities, only about 30% round correctly: roughly 17,600,000 are mis-rounded, and about 1,327,409 of those are off by more than one unit in the last place, the worst reaching about two units near the boundary between the two regions.

Carrying `r` at the `10^18` internal accumulation scale instead, with round-to-nearest in both the logarithm and square-root steps, realizes the precision the logarithm kernel already computes. The change is gas neutral: the `u256` operations are fixed cost, and the iteration counts of the logarithm and square-root kernels, the region split, the degrees, and the `u128` coefficient storage are all unchanged. Scored against an independent high-precision reference, this drops the tail region to 141,399 mis-rounded outputs (99.43% correctly rounded) and removes every output that was more than one unit off. Combined with a count-targeted re-

derivation of both regions, the whole function then reaches 99.00% correctly rounded, up from 92.16%, cutting its overall mis-rounded outputs from about 39,200,000 to 4,993,314. As with the coefficient re-derivation, this is the best result found rather than a proven optimum.

Consider computing and carrying the tail change of variable at the internal accumulation scale (10^{18}) with nearest rounding, rather than at the 10^9 output scale, so the tail rational receives an argument whose resolution matches the rest of the pipeline.

Update: Resolved in [pull request #483](#).

N-06 `inverse_cdf` Saturation Bound Is Inconsistent With `cdf`

The `MAX_Z_RAW` constant in `inverse_cdf_coefficients` is [documented](#) as the value returned when `inverse_cdf(1)` (and, reflected, `inverse_cdf(0)`) would otherwise need the unrepresentable $\pm\infty$, with the stated rationale that it "Matches the CDF domain bound so `cdf/inverse_cdf` agree at the corner". That claim does not hold. The constant is `MAX_Z_RAW = 6_300_000_000` ($z = 6.3$), while the CDF domain bound is `cdf_coefficients::max_z_raw() = 6_109_410_205` ($z = 6.109410205$), the analytic point where $\Phi(z)$ first rounds to `1.000000000` at nine digits. Since `cdf` returns `1.0` for every $z \geq 6.109410205$, the two functions disagree on where the representable domain ends: `inverse_cdf_upper_raw` saturates `inverse_cdf(1)` to $z = 6.3$, roughly `0.19` beyond the largest input the deployed `cdf` can still resolve to a value below `1.0`. The comment therefore documents an invariant that the code violates.

The round trip still holds, because `cdf(6.3)` saturates to `1.0`, and no representable probability maps into the `[6.109410205, 6.3]` interval through `cdf`. The concern is that two functions intended to be inverses of one another disagree at the corner, and that the coefficient module explicitly claims the opposite. Consider making the two bounds consistent by returning `cdf_coefficients::max_z_raw()` (`6.109410205`) from the `p = 1` branch, so the quantile saturates at the same $|z|$ the CDF treats as fully saturated, or, if a hard endpoint is preferred, aborting on `p = 1` rather than returning an out-of-range sentinel. Either way, the `MAX_Z_RAW` value and the comment asserting it matches the CDF bound should be brought back into agreement; the precise resolution is a developer decision. Whichever bound is chosen, the same `6.3` figure is repeated as the documented output range of the public entry points, in `math/fixed_point/README.md`'s "Standard-normal quantile" section and in the `#### Behavior` sections of `sd29x9_base::inverse_cdf` and `ud30x9_base::inverse_cdf`, and should be updated alongside `MAX_Z_RAW` so the public documentation does not go stale.

Update: Resolved in [pull request #478](#).

N-07 `WAD` Constant Uses 10^{36} Scale Inconsistently With Standard Meaning

The `cdf` and `pdf` modules define the internal fixed-point scale constant `WAD` as 10^{36} . In the established DSMath fixed-point convention, however, `wad` denotes 10^{18} (with `ray` denoting 10^{27} and `rad` denoting 10^{45}), and 10^{36} has no standard name under that convention. The same identifier is also used with its conventional 10^{18} value in `inverse_cdf` and in `horner_tests`, so `WAD` denotes two different magnitudes within a single package. This naming can mislead reviewers and integrators who expect the standard scale.

Consider renaming the 10^{36} constant to a neutral, self-describing name such as `SCALE`, `INTERNAL_SCALE`, or `PRECISION`, and reserving `WAD` for 10^{18} .

Update: Resolved in [pull request #479](#).

N-08 `cdf` and `pdf` Lack a Bit-Exact Regression Test Against Their Offline Codegen Mirror

The [oracle-comparison tests for `cdf`](#) and [for `pdf`](#) assert that the on-chain output matches a reference computed with `mpmath` at one hundred decimal digits of precision, within a five-ULP tolerance. This differs from the standard-normal quantile evaluator, where [tail_transform_matches_offline_mirror](#) asserts that the on-chain tail-transform kernel matches the offline Python integer mirror in `scripts/gaussian_codegen/shared/arithmetic.py` through exact equality.

A tolerance-based comparison against the mathematical oracle validates the accuracy of the AAA rational fit, but it does not by itself confirm that the on-chain Horner evaluation reproduces the offline reference arithmetic bit-for-bit; an implementation discrepancy between the two could in principle still pass a five-ULP tolerance check. No such discrepancy was identified during this audit: an independently constructed bit-exact replica of the on-chain arithmetic matched the true mathematical function well within the documented error budget across all evaluators. However, unlike the quantile's tail transform, `cdf` and `pdf` have no permanent, automated test enforcing bit-for-bit agreement with the offline mirror, leaving this specific property unguarded against future regressions.

Consider adding `cdf_matches_offline_mirror` and `pdf_matches_offline_mirror` tests, following the pattern established by `tail_transform_matches_offline_mirror`, to assert that the on-chain Horner evaluation matches the offline integer mirror through exact equality at a representative set of inputs.

Update: Resolved in [pull request #480](#).

N-09 Documentation Inconsistencies

Multiple instances of documentation that is inaccurate, internally inconsistent, or stale were identified across the Gaussian math sources, the offline codegen scripts, and the package documentation.

- The [monotonicity guarantee documented for pdf in sd29x9_base](#) describes the density as non-increasing "in $|z|$ " over the "at-risk tail ($|z| \geq 4$)". The [equivalent guarantee in ud30x9_base](#) describes the identical underlying property (both wrap the same `pdf_nonneg_raw`) using plain `z`. Since `UD30x9` values are non-negative, `z` and $|z|$ coincide, so the wording does not misstate behavior, but it is inconsistent with every other twin pair across the two modules.
- The [Aborts section of the horner_eval! macro](#) states that it "Aborts with `EEmptyPolynomial` if `$len == 0`". Every other `#### Aborts` bullet across the `math` and `contracts` packages states the constant and triggering condition directly, without an "Aborts with" prefix, since the heading already establishes that the section lists aborts. This is the only bullet in the codebase using that prefix.
- The [reference_values_float docstring in cdf/derive.py](#) ("nominal error is `3.35e-9`, three orders of magnitude above float64 precision") and the [reference.py module docstring](#) ("accurate to $\sim 1e-16$ - three orders of magnitude tighter than the `5e-9` error budget") misstate the margin: the gap between a $\sim 1e-16$ floor and a `3.35e-9 / 5e-9` target is about seven to eight orders of magnitude, not three.
- The [cdf_accumulation-scale comment](#) and the [pdf equivalent](#) describe the quantized output as "strictly monotone". Adjacent inputs frequently map to the same output, so it is only monotone (non-decreasing for Φ , non-increasing for ϕ), matching the "Monotone non-decreasing" wording the public base-module docs already use. The `pdf` phrase "strictly monotone non-increasing" is additionally self-contradictory.

- The [math package summary](#) lists [fixed_point](#) as providing "the standard-normal CDF ([cdf](#))" only, but [pdf](#) (density) and [inverse_cdf](#) (quantile) are now also public on both [UD30x9](#) and [SD29x9](#).
- The [cdf](#) and [pdf](#) accumulation-scale comments call 10^{36} "an order of magnitude finer" than the user-facing 10^9 ; the difference is 27 orders of magnitude.
- The [inverse_cdf module doc](#) motivates the domain split by stating the rational's "numerator and denominator both collapse toward zero" near $p = 1$. Since $\Phi^{-1}(p) \rightarrow +\infty$ there, the blow-up is a denominator-only pole (the numerator stays finite), and the reason for the split is that a low-degree rational cannot track the unbounded tail accurately.
- The [gaussian_codegen README](#) states the [inverse_cdf](#) tail transform "reuses the library's [ln/sqrt](#) kernels". It uses the internal [raw_log2](#) and [u256::sqrt](#) kernels instead ([ln](#) is derived from [log2](#)), deliberately not the typed [sd29x9_base::ln / sqrt](#).
- The [INVERSE_CDF_SPLIT doc](#) calls 0.975 "the classic Acklam/AS241 central-vs-tail split point". It is a chosen round value; Acklam's canonical break is 0.97575 , and AS241 uses different region boundaries.
- The [gaussian_codegen README overflow note](#) states "the peak Horner product clears 2^{256} ". The peak product stays under 2^{256} with headroom, so "clears" reads as the opposite of the intended meaning.

Consider correcting each docstring and comment above so it matches the exhaustively verified behavior and the current implementation, reusing the wording the correct twin already provides where one exists (for example, the [math/fixed_point/README.md](#) [6.3](#) rationale and the "Monotone non-decreasing" phrasing in the base modules).

Update: Resolved in [pull request #482](#).

Conclusion

The `openzeppelin_fp_math` package extends Sui's fixed-point decimal types, `UD30x9` and `SD29x9`, with closed-form standard-normal distribution functions (`cdf`, `pdf`, and `inverse_cdf`) evaluated as rational-function approximations generated by an offline Python pipeline. OpenZeppelin audited this Gaussian math addition together with its code-generation tooling and performed a diff audit of the surrounding fixed-point type API.

The internal architecture separates a small, reusable Horner-evaluation engine from per-function coefficient tables that are generated, validated, and drift-checked by dedicated offline tooling rather than derived on-chain, and the accompanying continuous-integration checks confirm the shipped coefficients are numerically sound and consistent with their recorded fit. Documentation of the approximation methodology and domain assumptions is largely complete but was found to be inconsistent or imprecise in a number of places across both the Move sources and the Python tooling, and the rational-evaluation logic shared by the three distribution functions is duplicated rather than factored into common code. The diff-audited base modules that wire these functions onto the fixed-point types introduce no changes to the existing arithmetic's rounding or overflow behavior.

The most substantive observations concern the design of the approximation itself: the coefficients are fit to minimize continuous approximation error rather than the count of fixed-point outputs that round incorrectly, and the tail region of `inverse_cdf` carries its change-of-variable computation at a coarser scale than its own kernels support. Neither was found to produce results outside the documented error budget, but each represents room for a materially tighter implementation. Two of the three distribution functions also lack a permanent bit-exact regression test against their offline arithmetic mirror, unlike the third, so strengthening that test coverage would help guard the numerical properties confirmed during this engagement against future regressions. No critical, high, medium, or low severity issues were identified; all findings were classified as informational notes.

The OpenZeppelin development team is thanked for their cooperation and responsiveness throughout the engagement.

Appendix

Independent Numerical Verification

To complement the pipeline's own continuous-integration gates, three numerical properties of the shipped `cdf`, `pdf`, and `inverse_cdf` were reproduced from scratch during this engagement: the true error against a high-precision reference, the absence of integer overflow, and the monotonicity of the output. The reproduction was kept independent of the codebase. The reference oracle is `mpmath`, a Python arbitrary-precision library, evaluated at 60 decimal places; the coefficients are parsed directly from the committed `.move` tables; the fixed-point pipeline is re-implemented from scratch in Python; and no repo-reported accuracy figure, committed test vector, or code-generation `max_error` value is relied upon at any point. Every representable input in each function's domain is classified exactly.

This appendix reports the behavior of the implementation as shipped at the audited commit.

Methodology

The verification proceeds in four parts.

- **Oracle.** The reference values come from `mpmath` at 60 decimal places: Φ from the error function, $\phi = \exp(-z^2/2)/\sqrt{(2\pi)}$, and Φ^{-1} from the inverse error function. Each shipped output is compared against the correctly rounded reference value $\text{round}(f(x) \cdot 10^9)$ on the public 10^9 grid.
- **Coefficients.** The rational coefficients are parsed directly out of the committed `cdf_coefficients`, `pdf_coefficients`, and `inverse_cdf_coefficients` modules and quantized so that they reproduce the committed `u128` integers exactly. Nothing is taken from the pipeline's reported error or from its intermediate fit files as evidence of accuracy.
- **Integer pipeline.** The on-chain evaluation is re-implemented independently: a sign-magnitude 256-bit Horner accumulation, with per-step floor division at the internal scale (10^{36} for `cdf` and `pdf`, 10^{18} for `inverse_cdf`), a final `mul_div` back to the 10^9 grid rounded to nearest, the saturation clamps at the analytic cutoffs, the reflection used by the signed type, and the two-region split of `inverse_cdf` with its logarithm and square-root tail change of variable. This re-implementation, not a mathematical

idealization, is what is scored: its output is the exact integer the on-chain function returns.

- **Exact classification of every input.** Every representable input in each domain is classified: 6,109,410,206 inputs for `cdf` (z in $[0, 6.109410205]$), 6,402,729,806 for `pdf` (z in $[0, 6.402729806]$), and 500,000,001 upper-half inputs for `inverse_cdf` (p in $[0.5, 1]$). The classification is exact, not sampled, and is made rigorous without recomputing 256-bit integers and 60-digit references at every one of billions of points, as follows. A fast float64 evaluation is used only to locate the inputs that sit close to a rounding boundary (where the output or the reference value is near a half-integer on the 10^9 grid). Every such candidate is then recomputed with the exact 256-bit integer pipeline and the `mpmath` reference, so its classification is exact. The float64 pass is trusted only away from those boundaries, and only because its error there is bounded far below the flag width: the float64 output error was measured on a dense grid and is smaller than the flag width by a factor of roughly 80 for `cdf` and 560 for `pdf`, so no input outside the flagged band can be on the wrong side of a rounding boundary. For `cdf` and `pdf` two facts make the result a proof rather than a spot check: the integer pipeline was confirmed to equal the correctly rounded value of its own rational at every flagged input (zero disagreements), and the finite set of grid points where even float64 evaluation of the reference disagrees with the exact reference was enumerated in full (364 points for `cdf`, 87 for `pdf`) and corrected. For `inverse_cdf`, whose central rational is more sensitive at the narrower 10^{18} scale, the float64 output is not accurate enough to certify by margin, so the integer output is instead computed exactly at all 500,000,001 inputs, with only the well-conditioned reference inverse left to the margin argument (its float64 error is below the flag width by a factor of about 315, and every near-boundary input is refined with `mpmath`).

True errors: accuracy over every input

Scoring the exact integer output of each function at every input against the correctly rounded oracle value gives the following.

`cdf` (Φ), degree 9/9, domain $[0, 6.109410205]$, 6,109,410,206 inputs:

error	count	share
correctly rounded (0 ULP)	5,894,399,747	96.4807%
+1 ULP	197,717,094	3.2363%
-1 ULP	17,293,365	0.2831%

error	count	share
beyond 1 ULP	0	0

`cdf` is never worse than 1 ULP anywhere on its domain.

`pdf` (Φ), degree 10/10, domain [0, 6.402729806], 6,402,729,806 inputs:

error	count	share
correctly rounded (0 ULP)	6,281,339,110	98.1041%
+1 ULP	82,124,492	1.2826%
-1 ULP	39,266,204	0.6133%
beyond 1 ULP	0	0

`pdf` is never worse than 1 ULP anywhere on its domain.

`inverse_cdf` (Φ^{-1}), two-region, upper half [0.5, 1], 500,000,001 inputs. Overall 92.16% correctly rounded, split by region:

region	inputs	correctly rounded	± 1 ULP	beyond 1 ULP
central ($p < 0.975$)	475,000,000	95.4583%	21,572,846	0
tail ($p \geq 0.975$)	25,000,000	29.4744%	16,303,992	1,327,409

The central region is within 1 ULP at every input. The tail carries all 1,327,409 of the beyond-1-ULP inputs, with a worst error of 2.083 ULP at the seam near $p \approx 0.975935$, all within the function's documented 5-ULP contract. The tail error is uniformly low-biased, a direct consequence of the downward-truncating logarithm and square-root change of variable used there.

No overflow

Replaying the exact Horner evaluation over the full domain and tracking the largest 256-bit intermediate product gives the peak value each function reaches.

function	peak 256-bit product	headroom below 2^{256}
<code>cdf</code>	$\approx 2^{245.9}$	≈ 10 bits

function	peak 256-bit product	headroom below 2^{256}
<code>pdf</code>	$\approx 2^{247.2}$	≈ 9 bits
<code>inverse_cdf</code>	$\approx 2^{130.3}$	≈ 126 bits

The peak for each function occurs at the top of its domain, where both the accumulator magnitude and the promoted input are largest. No intermediate value in any function can overflow the `u256` accumulator on any input in the scanned domain. The tightest case is `cdf`, which still leaves a 10-bit (roughly thousand-fold) margin.

Monotonicity maintained

Monotonicity is checked at input resolution, comparing every consecutive raw input rather than sampling a grid.

- `cdf`: 0 inversions over all neighbor pairs; the output is non-decreasing across the domain.
- `pdf`: 0 inversions over all neighbor pairs; the output is non-increasing.
- `inverse_cdf`: 0 inversions over all 500,000,001 pairs; the output is non-decreasing.

For `cdf` and `pdf` the internal accumulation scale of 10^{36} keeps the per-step floor-truncation noise below the true increment between neighbors, so no tail stutter occurs.

Cross-function consistency

Several identities that tie the three functions together were checked directly on the shipped implementation. $\Phi(0)$ equals exactly one half. $\phi(0)$ equals $1/\sqrt{2\pi} = 398,942,280$ exactly. The shipped `pdf` integrates to 1.00000000038 over the two-sided domain, and its cumulative sum tracks both the shipped `cdf` and the oracle Φ to about 10^{-10} across the domain; $\phi = \Phi'$ is confirmed by finite-differencing the on-chain `cdf`. The round trip `cdf(inverse_cdf(p))` was spot-checked across the quantile range and recovers p to within the combined rounding of the two functions, and the two `inverse_cdf` regions join continuously at the $p = 0.975$ seam.

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High

- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system. These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.